



Herramientas para facilitar la adaptación de la ICD-API de la OMS en el software de las IPS de Colombia

Transición de CIE-10 a CIE-11

Dirección de Epidemiología y Demografía
Grupo de gestión de conocimiento y fuentes de
información

Ministerio de Salud y Protección Social

Diciembre de 2024



Tabla de contenido

Objetivo.....	3
Alcance	3
Descripción de la ICD-API de la OMS	3
Contexto nacional.....	5
Aplicaciones Web	5
Aplicaciones de escritorio	6
Flujos de información	7
¿Como realizar la implementación?	8
Servicio de obtención del TOKEN	10
Instalación	11
Servicio de CIE-10 correspondiente	12
Instalación	13
Plantilla para aplicaciones WEB	15
Librería para BACKEND	16
Especificaciones para C#	21
Especificaciones para Java	22
Especificaciones para Python	22



Objetivo

Disponer a todas las IPS del país, herramientas que les faciliten el acceso e implementación de los servicios creados por la OMS incluidos en la ICD-API, para el uso de la CIE-11 dentro del Software para el registro de las historias clínicas.

Alcance

El presente documento contiene la explicación de cada una de las herramientas provistas por la OMS para facilitar su uso a través de la ICD-API, para la integración de la CIE-11 en las aplicaciones que utilizan las IPS del país para el registro de historias clínicas.

Estas herramientas facilitan el consumo de los servicios dispuestos, teniendo en cuenta los diferentes escenarios que se presentan en el país, dado que se utilizan diversas aplicaciones de software, construidas en diferentes arquitecturas y lenguajes de programación.

Por cada una de estas herramientas se dejan disponibles tanto ejemplos funcionales, para implementación directa, como el código fuente para que les permita a los desarrolladores de software realizar las modificaciones que crean convenientes.

Descripción de la ICD-API de la OMS

La OMS ha dispuesto una API REST basada en HTTP/(HTTPS) que permite el acceso mediante programación a la Clasificación Internacional de Enfermedades (CIE), dirigida a desarrolladores que deseen consumirla desde sus propios Software. Esto con el fin de permitir la integración de la clasificación dentro de las diferentes soluciones de software que se utilizan en los países para el registro de la información tanto de la mortalidad como de la morbilidad y facilitar la recolección de información de forma estándar.

Toda la información referente a la API se encuentra en la página <https://icd.who.int/icdapi>, donde se puede acceder a la documentación de esta, para entender su estructura, uso y funcionamiento. En esta página, se puede acceder a la documentación de referencia (Open-



API) por medio de la herramienta SWAGGER, donde se encuentran documentados cada uno de los métodos GET y POST existentes, además de los esquemas utilizados. Además de esta documentación, la OMS ha dispuesto un repositorio de GitHub donde se encuentran ejemplos de código fuente en diferentes lenguajes de programación (Ruby, Python, Java, PHP y C#), que ilustran la forma en la cual se puede consumir la API y facilita su implementación en el software.

Los aspectos más relevantes frente al uso de esta API son:

1. Se encuentra alojada en la infraestructura en la nube de la OMS, pero se tiene la posibilidad de realizar implementaciones en ambientes locales por medio de un contenedor Docker, un servicio de Windows o un servicio System de Linux. La recomendación del Ministerio de Salud y Protección Social – MSPS de Colombia, es que la API, se instale de manera local.
2. Para el consumo de la API que se encuentra en la infraestructura de la OMS, es necesario crear una cuenta de usuario en el sitio principal de la API, dado que utiliza OAuth2 para la autenticación y autorización. Cabe resaltar que en las implementaciones locales no es necesario realizar el proceso de autorización, a no ser que se implemente un servicio de autenticación y/o autorización.
3. Los servicios soportan tanto el formato JSON-LD como RDF/XML. Sin embargo, la OMS recomienda el uso de JSON-LD.
4. Los servicios son multi lenguajes. En este momento están disponibles en español, inglés, francés, árabe y chino, entre otros. El manejo del idioma se realiza por medio del header asignado en el llamado a la API.
5. Se tiene acceso tanto al componente fundamental como a la linealización de estadísticas de mortalidad y morbilidad. Cada uno de los elementos que las componen, son denominados entidades y son identificados de forma única por medio de una URI asignada. Se debe tener en cuenta que esta URI varía en el caso de utilizar una implementación local de la API.
6. Para facilitar la implementación dentro de las aplicaciones web, la OMS creó la denominada “*Embedded Coding Tool*”, que facilita a los desarrolladores embeber dentro de su código fuente de aplicaciones web por medio de JavaScript, las mismas funcionalidades que se ofrecen tanto en la herramienta de codificación como del navegador que se pueden acceder desde la página principal de la CIE-11 <https://icd.who.int/es/>.



Contexto nacional

En el país cada una de las instituciones que prestan servicios de salud (IPS) utilizan diferentes aplicaciones de software para realizar el registro de la historia clínica de los pacientes, incluso algunas instituciones todavía realizan el registro en papel, dado su tamaño o el número de pacientes atendidos. Las instituciones que utilizan software para realizar el registro de las historias clínicas lo pueden haber adquirido a una empresa especializada (tercero) o puede que lo hayan desarrollado a la medida por cuenta propia.

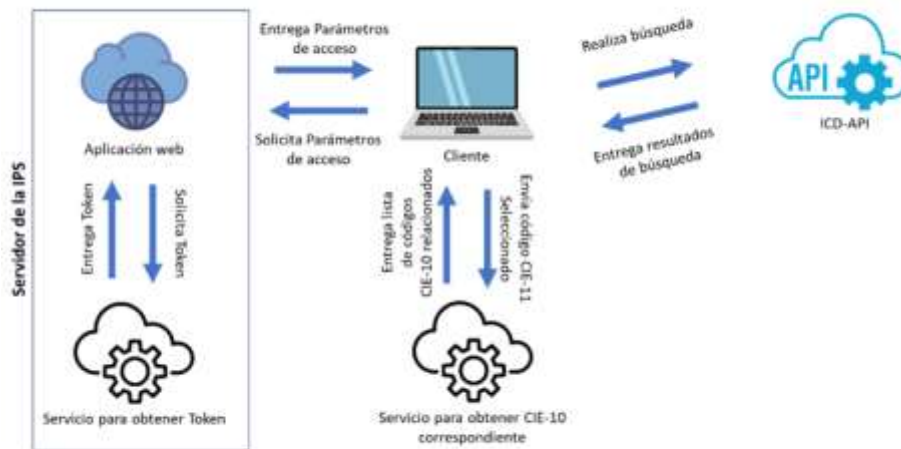
Dado lo anterior, se debe tener en cuenta que estas aplicaciones pueden utilizar diferentes arquitecturas de software, infraestructuras y lenguajes de programación, incluso los procesos de captura de datos pueden ser totalmente individualizados, lo que lleva a que cada software responda a la forma como están planteados dichos procesos y procedimientos.

Aplicaciones Web

En el país, existen IPS que utilizan para el registro de las historias clínicas aplicaciones web, las cuales utilizan como base lenguaje HTML y pueden integrar fácilmente funciones y métodos desarrollados en lenguajes de programación interpretados como JavaScript.

En este escenario se asume que los equipos que acceden como clientes a la aplicación, lo realizan por medio de un navegador web y cuentan con acceso a internet, por lo cual pueden establecer conexión directa con la ICD-API ya sea en la infraestructura dispuesta en la nube o en implementación local realizada en la infraestructura propia de cada IPS.





1

1

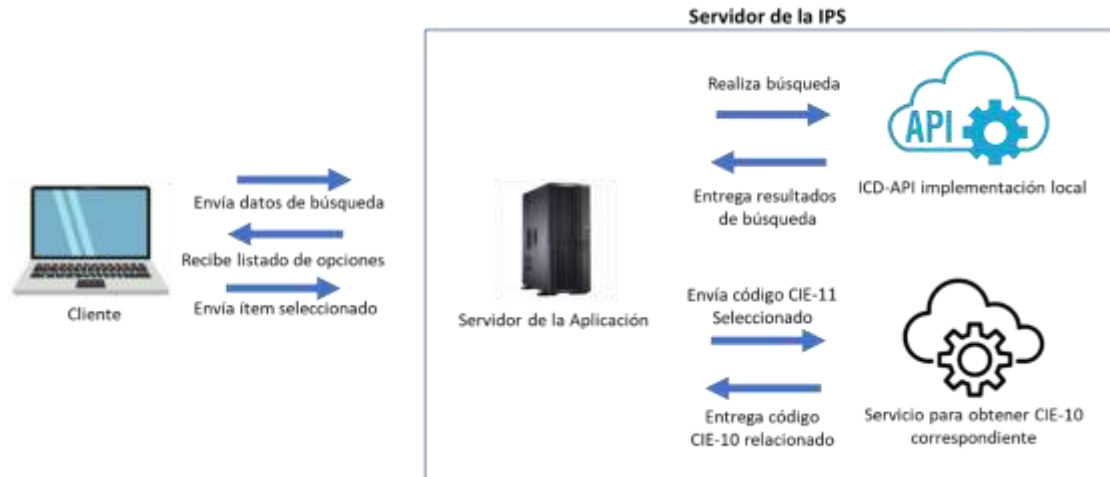
Aplicaciones de escritorio

Otro tipo de aplicaciones que se pueden encontrar para el registro de historias clínicas, son las de escritorio, es decir, aquellas que requieren la instalación de la aplicación en cada computador o equipo cliente.

Aquí se pueden encontrar aplicaciones que funcionen en el esquema cliente-servidor o aplicaciones que funcionan únicamente en el equipo, sin comunicarse con un servidor. En este escenario, ya no es posible utilizar la “*Embedded Coding Tool*” provista por la OMS, ya que las aplicaciones en la mayoría de las ocasiones utilizan lenguajes de programación compilados. Existen excepciones, tales como las aplicaciones desarrolladas en Node.js, que permiten utilizar código de JavaScript en una aplicación de escritorio.

En este escenario nos concentraremos en que se trabaja con lenguajes compilados y el acceso a la ICD-API se debe realizar desde el BackEnd, ya sea en aplicaciones locales o cliente-servidor. Como se muestra en la 2, existe la posibilidad que desde el equipo donde se controla la aplicación (es decir, el equipo local o el servidor), se tenga acceso a Red.

2La diferencia con la implementación web, es que se debe realizar el consumo de los servicios con el lenguaje de programación que se utilice para que todas las solicitudes y respuestas que se realicen y reciban de la ICD-API, las controle el servidor y este se los disponga a la interfaz gráfica de la aplicación cliente.



*Esquema de funcionamiento para aplicaciones de escritorio sin acceso a internet

En el caso que la aplicación no cuente con acceso a internet, tal como se muestra en la **¡Error! No se encuentra el origen de la referencia.**, se hace necesario que en la infraestructura con la que cuenta la IPS, se realice la implementación de la ICD-API de forma local, al igual que el servicio que permite obtener los CIE-10 correspondientes.

Flujos de información

Las IPS deben realizar el reporte de información para la consolidación de datos nacionales que se requiere para el Sistema General de Seguridad Social en Salud (SGSSS).

Lo anterior conlleva a la necesidad de la realización del registro dual de las clasificaciones, mientras se completa la transición. Sin embargo, previendo el gran esfuerzo adicional que tendrían los profesionales de la salud y/o los codificadores clínicos en la asignación de la clasificación tanto de la CIE-10 como de la CIE-11 para cada registro individual y basados en los insumos existentes, para Colombia se plantea esta dualidad desde un punto de vista diferente. Dado que se tienen mapeos disponibles entre las dos revisiones, estos fueron utilizados para construir una herramienta de software que permita asignarle el código CIE-10 correspondiente, una vez sea asignado o seleccionado el código CIE-11 dentro de los sistemas de información.



Esto permitirá que las instituciones prestadoras de servicios de salud almacenen y reporten desde sus aplicaciones de software tanto el código CIE-11 como el código CIE-10.

¿Como realizar la implementación?

Teniendo en cuenta lo explicado en las secciones anteriores y en especial los posibles escenarios que se pueden encontrar, para la modificación de los diversos Software se ha creado herramientas que facilitan el trabajo de desarrollo de software que se debe realizar en cada una de las aplicaciones, estandarizando la forma en la cual se realiza la consulta a la ICD-API y cómo se pueden utilizar los servicios de obtención de token y de obtención de los CIE-10 correspondientes.

Estas herramientas permiten concentrar el trabajo a realizar en la interfaz gráfica de las aplicaciones y ahorran tiempo en el proceso de consulta directa a los diferentes servicios, permitiendo la integración de todas las funcionalidades necesarias.

Con respecto a la interfaz gráfica de cada una de las aplicaciones, se recomienda que sean manejadas de forma similar a como se muestra en la herramienta de codificación adaptada para Colombia, publicada por el MSPS en la página <https://transicioncie11.sispro.gov.co/>, ya que allí se pueden observar las diferentes funcionalidades que se pueden implementar de acuerdo con la estructura de la CIE-11.

Se debe tener en cuenta que esta interfaz es funcional para realizar búsquedas dentro de la linealización de mortalidad y morbilidad de la CIE-11. Estas búsquedas se basan en el ingreso de un texto que es enviado a la ICD-API, para que encuentre los términos coincidentes y entregue el listado de categorías que coinciden con la búsqueda realizada. Para entender mejor la funcionalidad descrita, realice búsquedas de ejemplo en la herramienta de codificación que fue referenciada en el párrafo anterior.

Una vez se obtenga el listado de categorías que coinciden con la búsqueda, por cada una de ellas se pueden consultar sus detalles como la descripción, los términos que coinciden con la búsqueda (términos del índice), si tiene post-coordinación, información adicional, notas de codificación, entre otros (para conocer en detalle a que información se tiene acceso, revise la documentación del esquema “*LinearizationEntity*” en el API-Reference accesible en la página <https://id.who.int/swagger/index.html>). En especial con la post-coordinación, en la respuesta recibida por cada una de las categorías se conoce si se tiene o no post-coordinación disponible.



En la interfaz gráfica se debe dar la posibilidad de seleccionar la post-coordinación en el caso de estar disponible.

Por cada una de las categorías se pueden encontrar post-coordinaciones con uno o más ejes, lo cual se especifica en la respuesta recibida de la ICD-API. Esto se puede identificar con una variable obtenida llamada *“isLeaf”* que, en el caso de ser padre, tiene el valor *“false”*. En estos casos, dentro de la interfaz gráfica se debe dar la posibilidad de mostrar las categorías que son hijos de estas por medio de un árbol u otra estructura jerárquica.

Una vez se haya seleccionado por parte del usuario el código de la categoría de la CIE-11, ya sea solo o con alguna post-coordinación, este código se envía al servicio para la obtención de los CIE-10 correspondientes, el cual entregará un listado de las categorías de la CIE-10 que están asociados al código de la categoría de la CIE-11 enviado, de acuerdo con los mapeos disponibles. Cabe aclarar que, en el caso de los códigos post-coordinados, en la respuesta obtenida se relacionan todas las categorías de la CIE-10 que se encuentren de los códigos base que hagan parte de la post-coordinación.

Dentro de la interfaz gráfica, se debe incluir este listado para que el usuario pueda seleccionar una y solo una de las categorías de la CIE-10. Se debe tener en cuenta que no todos los códigos CIE-11 tienen mapeos asignados con la CIE-10, por lo que se debe dar la opción al usuario de buscar la categoría CIE-10 dentro de la lista tabular, tal como lo ha venido realizando el software antes de esta modificación. Esta opción se debe dar en el caso que no existan mapeos disponibles o como posibilidad por si el profesional de la salud no encuentra un mapeo que coincida dentro del listado entregado.

La aplicación debe almacenar tanto la categoría del CIE-11 como la categoría de la CIE-10 que fueron seleccionadas por el usuario. Dado que el listado de categorías de la CIE-11 no se encuentra almacenado en la aplicación de registro de historias clínicas, también se debe almacenar el título de la categoría y la URI asignada para posteriores posibles búsquedas. El almacenamiento de datos adicionales dentro de la aplicación se deja a criterio de la IPS y el equipo de desarrollo de software. Teniendo en cuenta la estructura y taxonomía del sistema de clasificación de la CIE, no se recomienda que dentro de las aplicaciones se utilice simplemente la lista tabular como una lista desplegable, varias de ellas enlazadas o búsquedas en las palabras de los títulos de las categorías, dado que se pierde la gran utilidad que tiene el componente fundamental, el potencial de búsqueda que se tiene con los términos indexados y la posibilidad de post-coordinación.

A continuación, se describen cada una de las herramientas que han sido desarrolladas de forma local para Colombia, que facilitan realizar la implementación descrita en este apartado.



Servicio de obtención del TOKEN

Este servicio se ha desarrollado como una API REST basada en HTTP, que permite la obtención automática de un token válido para realizar consultas a la ICD-API alojada en la infraestructura dispuesta por la OMS en la nube, dado que se utiliza el protocolo de autorización OAuth 2 (para más información puede consultar la página <https://icd.who.int/icdapi/docs2/APIIDoc-Version2/#authentication>). Este servicio fue desarrollado utilizando el framework .NET Core de Microsoft, que permite ser implementado en diferentes sistemas operativos y servidores web.

La versión publicada por el MSPS, funciona con la versión 9.0 o superior del framework. (Entorno para publicación web: <https://download.visualstudio.microsoft.com/download/pr/e1ae9d41-3faf-4755-ac27-b24e84eef3d1/5e3a24eb8c1a12272ea1fe126d17dfca/dotnet-hosting-9.0.0-win.exe>;

SDK para compilar:

<https://download.visualstudio.microsoft.com/download/pr/38e45a81-a6a4-4a37-a986-bc46be78db16/33e64c0966ebdf0088d1a2b6597f62e5/dotnet-sdk-9.0.101-win-x64.exe>.

Puede verse como un microservicio, dado que solo cuenta con un Endpoint que permite consultar un token válido. Este servicio tiene en cuenta las políticas CORS, permitiendo que sea consumido desde cualquier origen. Debe tener en cuenta que, para el correcto funcionamiento del servicio, debe tener un certificado, ya que tiene habilitado el cifrado con SSL.

Dado que, para la obtención del token se debe contar con un **client ID** y un **client Secret**, este servicio los lee de un archivo en formato JSON que se encuentra en la carpeta **Data**, por lo cual se debe asegurar que no se permita el acceso a las carpetas por medio del navegador.

Para realizar el consumo de esta API, se tiene un Endpoint tipo GET que se accede utilizando la URI <https://<dominio:puerto>/api/tokens>, que entrega un arreglo de objetos en formato JSON, con un solo elemento que contiene la fecha de solicitud del token y el valor de este. Para conocer más, puede revisar la documentación en formato Open-API (swagger) una vez lo implemente en el servidor web, ingresando por medio de un navegador web a la dirección <https://<dominio:puerto>/swagger>.

Se puede acceder a los instaladores de este servicio para ser utilizado tal como está estructurado o también acceder al código fuente para realizar los cambios que se crean convenientes tales como el cifrado de los datos de autenticación, modificaciones en la



política CORS u otras configuraciones que se consideren necesarias de acuerdo con sus políticas de software.

También se puede utilizar este código fuente como guía, en el caso que se decida desarrollar su propio servicio.

Instalación

Para realizar la implementación de este servicio, debe instalar el framework .NET Core de Microsoft. Esta instalación depende del sistema operativo que se utilice, por lo cual debe consultar la documentación que se encuentra en la página <https://docs.microsoft.com/es-es/dotnet/core/install/>.

Para la implementación como tal del servicio, puede consultar la documentación de Microsoft contenida en la página <https://docs.microsoft.com/en-us/aspnet/core/host-and-deploy/?view=aspnetcore-9.0>, donde encuentra los diferentes escenarios según la infraestructura en la cual se vaya a hospedar.

Una vez se encuentre implementado correctamente el servicio, para su correcto funcionamiento, debe entrar a la carpeta **Data** donde se encuentra un archivo con extensión **Json** llamado **securefile**. Abra este archivo en un editor de texto como Notepad y coloque los datos correspondientes para el **client ID** y **client Secret**, que obtuvo en la página de la OMS.

Para garantizar que el servicio siempre se encuentre activo, dependiendo del servidor web utilizado, se debe configurar que el servicio este siempre disponible y no por demanda

Compruebe el funcionamiento iniciando el servicio y entrando a un navegador, donde debe entrar en la URL <https://<dominio:puerto>/api/tokens>. Allí se mostrará el token que se ha obtenido. Al ser la primera vez que se ejecuta el servicio, puede demorarse unos segundos.

Si hace uso de un proxy o un firewall, se debe realizar la configuración necesaria y comprobar que se tiene acceso a la URL <https://icdaccessmanagement.who.int>.

Servicio de CIE-10 correspondiente

Este servicio se ha desarrollado como una API REST basada en HTTP, que permite obtener las categorías CIE-10 correspondientes que tiene asociadas un código de las categorías de



la CIE-11, de acuerdo con los mapeos disponibles que ha publicado la OMS en la página <https://www.minsalud.gov.co/sites/rid/Lists/BibliotecaDigital/RIDE/VS/ED/GCFl/mapping-cie10-cie-11.zip>

También permite obtener en formato JSON todos los mapeos disponibles. Este servicio fue desarrollado utilizando el framework .NET Core de Microsoft, que permite ser implementado en diferentes sistemas operativos y servidores web. Funciona con la versión 9.0 del framework.

Esta API contiene 2 Endpoint tipo GET, que se encuentran documentados en el enlace <https://<dominio:puerto>/swagger/index.html>:

1. **/api/Mappings**: Permite obtener todos los mapeos disponibles.
2. **/api/Mappings/{icd11Code}**: Permite obtener las categorías CIE-10 correspondientes, de un código de categoría de la CIE-11 que se entregue como parámetro **{icd11Code}**. Se debe tener en cuenta que como parámetro se puede enviar un código post-coordinado y se recibirá como respuesta las categorías CIE-10 correspondientes de todos los códigos base que se encuentren. En el caso de códigos post-coordinados se debe reemplazar el carácter “/” por “-”.

La respuesta obtenida es una lista de elementos en formato JSON, donde cada uno contiene el código y nombre de la categoría CIE-10.

Para la implementación de forma local, se puede acceder a los instaladores de este servicio para ser utilizado tal como está estructurado o también acceder al código fuente para realizar los cambios que se crean convenientes tales como modificaciones en la política CORS u otras configuraciones que se consideren necesarias de acuerdo con sus políticas de software.

Para su funcionamiento, dentro de la carpeta **Data**, se encuentra un archivo en formato **Json** llamado **mapping11To10**. En el caso que se realice una actualización de los mapeos existentes, el MSPS avisará oportunamente y pondrá disponible el archivo para realizar la actualización de la implementación local.

Instalación

Para realizar la implementación de este servicio de forma local, debe instalar el framework.NET Core 9 de Microsoft. Esta instalación depende del sistema operativo que se utilice en el servidor de instalación, por lo cual debe consultar la documentación que se encuentra en la página <https://docs.microsoft.com/es-es/dotnet/core/install/>



Para publicar aplicaciones Web sobre plataforma Windows, se recomienda el Hosting Bundle: <https://dotnet.microsoft.com/en-us/download/dotnet/thank-you/runtime-aspnetcore-9.0.0-windows-hosting-bundle-installer>. Descargue el paquete y siga las instrucciones del instalador.

Descargue de las publicaciones compartidas del MSPS, la versión a utilizar:

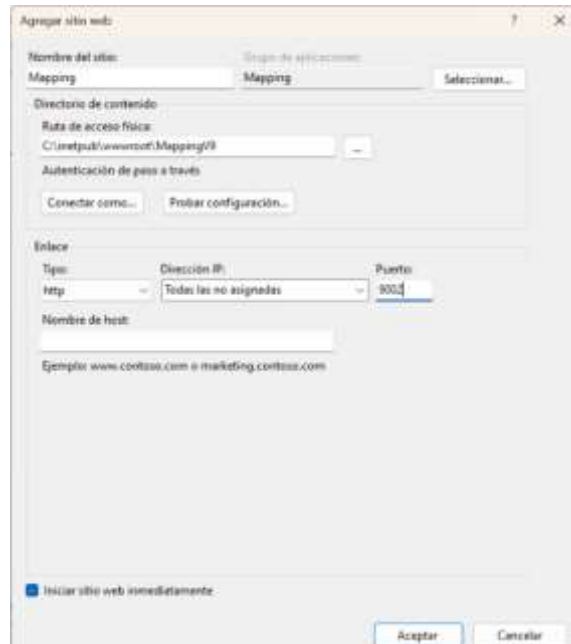
<https://www.minsalud.gov.co/sites/rid/Lists/BibliotecaDigital/RIDE/VS/ED/GCFI/api-mapping.zip>

Posteriormente, ingrese a la consola del Internet Information Server (IIS). Sobre la opción Sitios y con el botón derecho haga clic, después seleccione el sub menú Agregar sitio web

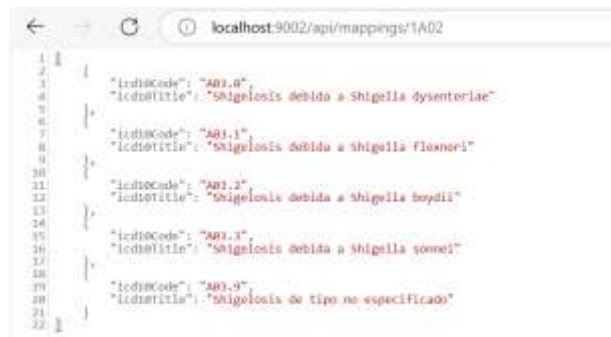


Ingresa los datos para la publicación del nuevo sitio web

- El nombre del sitio web
- La ruta donde se encuentran los archivos del nuevo sitio web
- El puerto en el cual se publicará el nuevo sitio



Compruebe el funcionamiento iniciando el servicio y entrando a un navegador, donde debe entrar en la URL <https://<dominio:puerto>/api/Mappings/1A02>. Allí se mostrará el listado de categorías asociadas al código 1A02 de la CIE-11 que se ha obtenido. Al ser la primera vez que se ejecuta el servicio, puede demorarse unos segundos.



```

1  {
2    "id": "1A02",
3    "title": "Shigellosis debida a Shigella dysenteriae",
4    "description": "Shigellosis debida a Shigella dysenteriae",
5    "category": "Shigellosis",
6    "subCategory": "Shigellosis debida a Shigella dysenteriae",
7    "id": "1A02.1",
8    "title": "Shigellosis debida a Shigella flexneri",
9    "description": "Shigellosis debida a Shigella flexneri",
10   "category": "Shigellosis",
11   "subCategory": "Shigellosis debida a Shigella flexneri",
12   "id": "1A02.2",
13   "title": "Shigellosis debida a Shigella boydii",
14   "description": "Shigellosis debida a Shigella boydii",
15   "category": "Shigellosis",
16   "subCategory": "Shigellosis debida a Shigella boydii",
17   "id": "1A02.3",
18   "title": "Shigellosis debida a Shigella sonnei",
19   "description": "Shigellosis debida a Shigella sonnei",
20   "category": "Shigellosis",
21   "subCategory": "Shigellosis debida a Shigella sonnei",
22   "id": "1A02.9",
23   "title": "Shigellosis de tipo no especificado",
24   "description": "Shigellosis de tipo no especificado",
25   "category": "Shigellosis",
26   "subCategory": "Shigellosis de tipo no especificado"
27 }

```

Si hace uso de un proxy o un firewall, se debe realizar la configuración necesaria para que la aplicación local tenga acceso a este servicio.



Plantilla para aplicaciones WEB

Para que se tenga acceso a un ejemplo funcional de cómo se puede realizar la implementación de la CIE-11 dentro de las aplicaciones web, se dispone una plantilla basada en la herramienta “*Embedded Coding Tool*” <https://www.minsalud.gov.co/sites/rid/Lists/BibliotecaDigital/RIDE/VS/ED/GCFI/plantilla-ejemplo-frontend-cie10-cie11.zip>, o podrá utilizar la realizada como herramienta adaptada para Colombia de la página <https://transicioncie11.sispro.gov.co/>.

Por otra parte, también está disponible, la dispuesta por la OMS (para más detalles puede consultar la página <https://icd.who.int/icdapi/docs2/icd11ect-1.4-EmbeddedCodingTool/>).

En términos generales, la *Embedded Coding Tool*, utiliza una librería llamada “*Embedded Classification Tool (ECT)*” desarrollada en JavaScript, que permite acceder a los diferentes Endpoint de la ICD-API de forma fácil.

La librería necesita 2 parámetros para su funcionamiento:

1. **mySettings:** Contiene los atributos necesarios para la conexión. El único obligatorio es la URL de la API. Para más detalles puede consultar en <https://icd.who.int/icdapi/docs2/icd11ect-1.4-EmbeddedCodingTool/#settings>.
2. **myCallbacks:** Contiene los métodos necesarios para estructurar las acciones que se deben realizar en eventos específicos. Para más detalles puede consultar en <https://icd.who.int/icdapi/docs2/icd11ect-1.4-EmbeddedCodingTool/#callbacks>.

Específicamente, en la plantilla creada se coloca como ejemplo en uno de los Callbacks llamado `selectedEntityFunction`, las acciones para tomar la entidad seleccionada y mostrar los datos del código, título y descripción del texto que coincide mejor con la búsqueda, en cuadros de texto.

También se realiza el llamado a la API para obtener las categorías CIE-10 correspondientes basados en el código seleccionado. Los resultados se muestran en una lista de opciones de selección única (radio buttons). Dentro de esta plantilla también se da la opción de seleccionar algún elemento de la lista o realizar la búsqueda de forma manual en el caso que no existan mapeos o el profesional de la salud no encuentre una opción que coincida con su diagnóstico.

En el Callback llamado `getNewTokenFunction` se muestra un ejemplo de cómo se realiza el llamado a la API para obtener el token y cómo leer el resultado, para que sea incluido en el encabezado de la solicitud a la ICD-API dispuesta en la infraestructura de la OMS.



Para facilitar el acceso a los elementos HTML y la llamada a las API, se utiliza dentro de la plantilla JQuery. Sin embargo, se pueden utilizar las herramientas y técnicas que estén acordes a la metodología de desarrollo que se utilice.

El ejemplo planteado se realiza teniendo en cuenta que sólo se tenga una instancia de la herramienta de codificación. Si es necesario manejar más de una de ellas, puede consultar la documentación en <https://icd.who.int/icdapi/docs2/icd11ect-1.4-AdvancedSettings/#multiple-instances-of-embedded-classification-tool-components>.

Librería para BACKEND

Dado que todas las aplicaciones existentes en el país no son web y no pueden utilizar la Embedded Coding Tool, se crea una librería que permita acceder a los datos necesarios para crear la interfaz gráfica que integre la CIE-11 en las aplicaciones de escritorio que tengan o no acceso a internet y que utilicen o no un servidor para su funcionamiento, tal como se explicó en la sección **Aplicaciones de escritorio** del presente documento.

Esta librería está compuesta por 6 clases, que permiten acceder a los Endpoint de la ICD-API, a las API creadas (obtener token y obtener las CIE-10 correspondientes) y estructurar los datos para que se puedan utilizar en la implementación que se realice dentro de la interfaz gráfica. Tanto los atributos como los métodos que las componen tienen acceso público, para que puedan ser utilizados libremente.

Cada una de ellas se describe a continuación:

APIconnection: Está compuesta por los atributos y métodos necesarios para comunicarse con las API dispuestas (La ICD-API en la infraestructura de la OMS o implementada de forma local, obtener token y obtener las CIE-10 correspondientes).

Esta clase contiene los siguientes atributos:

- **APIVersion:** Versión de la ICD-API en la que se van a realizar las consultas. Para más información consulte <https://icd.who.int/icdapi/docs2/APIDoc-Version2/>.
- **Language:** Idioma en el cual se va a realizar la consulta. Se deja como atributo asignable, dado que en algún escenario pueda requerirse realizar la consulta en un idioma diferente al español. Para más información consulte <https://icd.who.int/icdapi/docs2/SupportedClassifications/>.
- **URLAPI:** URL donde se encuentra alojada la ICD-API, ya sea en la infraestructura dispuesta por la OMS o en una implementación local. Se debe tener en cuenta que no debe terminar con el carácter "/", dado que se utiliza para realizar las consultas en diferentes Endpoints.



- **WithSecurity:** Este atributo es booleano y se debe asignar valor **true** en el caso que se necesite utilizar la API para obtener el token, es decir, en el caso que se esté comunicando con la ICD-API que se encuentra en la infraestructura de la OMS. Se debe asignar **false** en caso contrario.
- **URLSecurity:** URL de donde se encuentra alojado el servicio para la obtención de token. En el caso que no sea necesario, es decir, cuando WithSecurity = false, puede no asignarse. Se debe tener en cuenta que no debe terminar con el carácter “/”.
- **URLMapping:** URL de donde se encuentra alojado el servicio para la obtención de las categorías CIE-10 correspondientes, ya sea en la infraestructura del Ministerio de Salud o implementada localmente. Se debe tener en cuenta que no debe terminar con el carácter “/”.
- **Error:** Atributo booleano que permite conocer si se generó un error en la búsqueda realizada en el Endpoint de búsqueda por texto de la ICD-API (Para más información consulte la documentación de la API-Reference en <https://id.who.int/swagger/index.html>, específicamente para el Endpoint /icd/release/11/{releaseId}/{linearizationname}/search)
- **ErrorMessage:** En el caso que exista un error, este atributo contiene el mensaje recibido del error generado (Para más información consulte la documentación de la API-Reference en <https://id.who.int/swagger/index.html>, específicamente para el Endpoint /icd/release/11/{releaseId}/{linearizationname}/search).

Además, contiene los siguientes métodos:

- **GetToken:** Método que consulta la API para obtener el token y entrega como resultado el valor de un token válido para realizar consultas en la ICD-API dispuesta en la infraestructura de la OMS. Este método es el único que se deja protegido dentro de la clase.
- **CallAPI:** Método que permite realizar solicitudes al ICD-API en los diferentes Endpoints que la conforman y obtener la respuesta, para entregarlas en formato JSON a otros métodos específicos.
- **CallMapping:** Método que consulta la API para obtener el listado de categorías CIE-10 correspondientes de un código CIE-11 específico. Entrega un listado de categorías CIE-10 en el formato JSON tal como se obtiene de la API.

Category: Está compuesta por los atributos necesarios para recibir la respuesta obtenida del Endpoint /icd/release/11/{releaseId}/{linearizationname}/search de la ICD-API, específicamente de las categorías encontradas. Los atributos son:

- **Id:** URI de identificación de la entidad que fue recibida como coincidencia.
- **Code:** Código de la categoría de la CIE-11 obtenida en la búsqueda.
- **Title:** Título de la categoría de la CIE-11 obtenida en la búsqueda.
- **StemId:** URI de identificación de la entidad principal que fue recibida como coincidencia, en el caso que se tenga poscoordinación.



- **IsLeaf:** Atributo booleano que permite identificar si la categoría es una hoja dentro del árbol de categorías. Es útil para saber si tiene categorías hijo.
- **IsResidualOther:** Atributo booleano que permite identificar si la categoría es residual de tipo “Otros”. Para más información, consulte la Guía de referencia de la CIE-11, en la sección “2.8.5 Categorías residuales – Otros y Sin especificación”.
- **IsResidualUnspecified:** Atributo booleano que permite identificar si la categoría es residual de tipo “Sin especificación”. Para más información, consulte la Guía de referencia de la CIE-11, en la sección “2.8.5 Categorías residuales – Otros y Sin especificación”.
- **TitleIsASearchResult:** Atributo booleano que permite identificar si la búsqueda coincide con el título de la categoría.
- **Chapter:** Capítulo al que pertenece la categoría.
- **Important:** Atributo booleano que permite identificar si el resultado tiene un nivel de importancia frente a los otros recibidos.
- **PostcoordinationAvailability:** Atributo que permite conocer si la categoría tiene o no poscoordinación. Tiene 3 posibles valores: 0 en el caso que no tenga poscoordinación disponible, 1 en el caso que tenga poscoordinación disponible pero que no sea obligatoria y 2 en el caso que tenga poscoordinación disponible

CIE10: Está compuesta por los atributos necesarios para recibir las categorías CIE-10. Los atributos son:

- **Icd10Code:** Atributo que contiene el código de la categoría CIE-10.
- **Icd10Title:** Atributo que contiene el título de la categoría CIE-10.

Linealization: Está compuesta por los atributos necesarios para identificar la linealización que se desea consultar. Estos atributos de dejan disponibles para asignación, dado que podría utilizarse esta clase para consultar otras linealizaciones de la CIE-11 o del componente fundamental (Para más información, consulte <https://icd.who.int/icdapi/docs2/APIDoc-Version2/#scope>).

También contiene métodos específicos que permiten comunicarse con Endpoints específicos para linealizaciones (Para más información consulte los Endpoints de linealización en <https://id.who.int/swagger/index.html>).

Esta clase contiene los siguientes atributos:

- **Name:** Nombre de la linealización en su versión resumida. Actualmente, la linealización para estadísticas para mortalidad y morbilidad está identificado con el nombre **mms**. Para las implementaciones necesarias en los Software de las IPS, se utiliza esta linealización, sin embargo, se deja abierto para que funcione la misma clase para otras linealizaciones.
- **ReleaseId:** Identificador de la liberación que se haya realizado de la linealización. Para utilizar la información más actualizada, se debe asignar el identificador de la



última liberación realizada, que actualmente es la 2021-05 (Para más información, puede consultar en <https://icd.who.int/icdapi/docs2/icd11ect-1.4-EmbeddedBrowser/#multilingual-support>).

- Connection: Atributo de tipo **APIconnection** que permite a la clase comunicarse con las API disponibles.

Además, contiene los siguientes métodos:

- SearchByTerm: Método que permite realizar una búsqueda por un texto específico, para que sea consultado en el índice de la CIE-11. Este método permite generar el complemento necesario para utilizar el Endpoint `"/icd/release/11/{releaseId}/{linearizationname}/search"`, recibiendo como parámetros, todos los contemplados para este Endpoint (Para más información, consulte la documentación del Endpoint en <https://id.who.int/swagger/index.html>). Este método permite sobrecarga, es decir, solo es obligatorio entregar el parámetro QueryText. Los demás parámetros pueden ser asignados en caso de ser necesario y tienen por defecto el valor que se especifica en la documentación.

Como respuesta de este método, se recibe una lista de elementos tipo **Category**, que se deben utilizar dentro de la interfaz gráfica como los resultados obtenidos de la búsqueda.

- SearchByCode: Método que permite realizar una búsqueda por un código específico de la CIE-11, el cual recibe como parámetro. Soporta el ingreso de códigos poscoordinados, por lo cual entrega un listado de entidades tipo LinearizationEntity, que corresponden ya sea al código de la categoría única o a los códigos base que se encuentren poscoordinados.
- GetMapping: Método que permite realizar una búsqueda de los códigos CIE-10 correspondientes a un código CIE-11 en la API dispuesta para este fin. Como parámetro recibe un código de la CIE-11 y entrega un listado de entidades de tipo **CIE10**.

Soporta el ingreso de códigos poscoordinados, realizando el cambio del carácter "/" por "-", con el fin de entregar el parámetro en el formato correcto, de acuerdo con la especificación el servicio.

LinearizationEntity: Está compuesta por los mismos atributos definidos para el esquema *LinearizationEntity* dentro de la ICD-API (consultar estructura del esquema en <https://id.who.int/swagger/index.html>), haciendo los siguientes cambios:



- Los atributos Title, Definition, LongDefinition, FullySpecifiedName y CodingNote, no mantienen la estructura del esquema *LanguageSpecificText*. Para estos atributos se toma directamente el texto del lenguaje dado.
- Los atributos child, parent, foundationChildElsewhere, indexTerm, inclusion, exclusion y postcoordinationScale, se mantienen en el formato JSON, para conservar la estructura que se tiene en la ICD-API.
- Contiene un atributo adicional llamado Connection de tipo APIconnection que permite que esta clase se comunique con las API disponibles.

Además, contiene los siguientes métodos:

- SearchById: Método que permite consultar una entidad de la linealización, recibiendo como parámetro la URI correspondiente. Es un procedimiento que permite asignar los valores de cada uno de los atributos de la clase, en un formato correcto, validando si los atributos si se encuentran en la respuesta recibida.
- GetChilds: Método que permite obtener un listado de entidades tipo **LinearizationEntity**, de los hijos que tenga la categoría específica. Esto se debe utilizar para crear los árboles de categorías en el momento que se tengan poscoordinaciones disponibles y estas categorías tengan categorías hijo.
- GetPostcoordination: Método que permite obtener las poscoordinaciones disponibles para la entidad específica. Como resultado se obtiene un listado de tipo PostcoordinationInfo, que especifica los ejes de posible poscoordinación.

PostcoordinationInfo: Contiene los atributos necesarios para caracterizar las poscoordinaciones disponibles para una entidad específica. Se basa en la estructura del esquema PostcoordinationScaleInfo dentro de la ICD-API (consultar estructura del esquema en <https://id.who.int/swagger/index.html>), haciendo los siguientes cambios:

- El atributo axisName en el esquema de la ICD-API, contiene una URI que identifica el eje. Dentro de esta clase se crean 2 atributos basados en este: AxisURL que contiene la URI recibida y AxisName que contiene como tal el nombre del eje para ser utilizado en la interfaz gráfica.
- Las URI que son recibidas en el atributo scaleEntity del esquema de la ICD-API, se almacena como un arreglo de cadenas de texto (string) en el atributo URIScales. Basados en estas URI, se obtiene con el método GetPostcoordination de la clase **LinearizationEntity**, el listado de entidades con las que se puede hacer poscoordinación de la entidad específica, por cada uno de los ejes.



Con estas clases y los métodos que las componen, se puede acceder a toda la información necesaria para construir una interfaz gráfica como la propuesta en el apartado “*Forma de implementación*” del presente documento.

Esta librería se encuentra desarrollada en los lenguajes de programación C#, Java y Python. En los casos de C# y Java, se dispone de la librería compilada. Para los 3 lenguajes también se deja disponible el código fuente. Adicional a esto, se deja disponible un ejemplo que permite probar la librería en una aplicación para ejecutar por consola.

En el caso de utilizar un lenguaje de programación diferente a los mencionados, debe descargar el código fuente de la librería, en el lenguaje de programación que tenga mayor experiencia y realizar la traducción al lenguaje de programación que se utiliza en la aplicación del a IPS.

Si cree necesario agregar nuevas clases y métodos a la clase, puede basarse en el código fuente suministrado en el lenguaje de su elección y desarrollar su propia versión de la librería. En este caso, debe tener en cuenta todas las recomendaciones, sugerencias y obligaciones que se tienen en el uso de la ICD-API, de acuerdo con la documentación existente y dispuesta por la OMS en <https://icd.who.int/icdapi>.

Especificaciones para C#

Para el uso de la librería traducida a C#, se debe realizar la instalación dentro de las dependencias del proyecto un paquete NuGet llamado **ICD11_CO**, que se deja disponible. Este paquete incluye las librerías externas IdentityModel y Newtonsoft.Json, lo que permite que se realice una única instalación, ya que instala automáticamente las dependencias.

La librería cuenta con todas las clases descritas en la sección anterior, teniendo en cuenta que los atributos que almacenan datos en formato JSON, se establecen de tipo dynamic para tener acceso directamente a los atributos de los elementos, teniendo en cuenta la estructura definida en los esquemas. Esto aplica sobre todo a los atributos de la clase **LinearizationEntity**.

El ejemplo de código fuente que se deja disponible para la prueba de la librería, contiene una solución compuesta por 2 proyectos: Código fuente de la librería y código fuente de la aplicación de consola. Esta solución puede ser abierta desde Visual Studio en cualquiera de sus versiones.



Especificaciones para Java

Para el uso de la librería en traducida a Java, se deja disponible un paquete llamado **ICD11_CO**, que contiene todas las clases descritas. Este paquete se deja disponible como un archivo con extensión **.jar**, para que pueda ser incluido fácilmente en las aplicaciones. Este paquete tiene una dependencia con el paquete **org.json**.

Estos dos paquetes deben ser incluidos dentro de la aplicación en el classPath y/o en el manifiesto, para acceder a todas las clases.

La librería cuenta con todas las clases descritas, teniendo en cuenta que los atributos que almacenan datos en formato JSON, se establecen de tipo JSONObject o JSONArray, de acuerdo con la respuesta recibida, para tener acceso directamente a los atributos de los elementos, teniendo en cuenta la estructura definida en los esquemas. Esto aplica sobre todo a los atributos de la clase **LinearizationEntity**.

El ejemplo de código fuente que se deja disponible para la prueba de la librería, contiene el código fuente de las clases en la carpeta **src**, el código fuente de la aplicación de consola en la carpeta **test**, una copia de la librería **org.json** en la carpeta **lib** (por si se requiere) y la librería compilada en la carpeta **dist**. Para el desarrollo de la librería, se utilizó como IDE de desarrollo NetBeans.

Especificaciones para Python

Para el uso de la librería en traducida a Python, se deja disponible un paquete llamado **ICD11_CO**, que contiene todas las clases descritas. Se debe tener presente que se crearon clases como tal y no solo métodos importables como módulos.

Esto es importante a la hora de realizar la importación, dado que es necesario especificar tanto el nombre del paquete como el nombre del archivo para importar la clase. Por ejemplo, para usar la clase **APIConnection**, se debe realizar la importación colocando **from icd_11_co.APIConnection import APIConnection**.

Este paquete tiene una dependencia con una librería llamada requests, que debe ser instalada en el ambiente de Python.

Se debe tener en cuenta que cada una de las clases tiene parámetros definidos, a excepción de **LinearizationEntity**. Esto es importante en el momento de utilizar las clases, porque exige que se instancien con todos los atributos.



El ejemplo de código fuente que se deja disponible para la prueba de la librería, contiene el código fuente de las clases y también el código fuente de la aplicación de consola. Para facilitar su uso, se deja en el espacio de trabajo, el paquete como una carpeta. Para el desarrollo de la librería, se utilizó como IDE de desarrollo Visual Studio Code.